Below is a structured lab guide with **sub-questions, Snort rules (custom messages), and Kali Linux test commands** for each requirement.

Assumptions

- Web Server: `192.168.74.10`
- Kali: `192.168.74.20`
- HOME_NET = Web Server
- EXTERNAL_NET = any
- Local rules file: `/etc/snort/rules/local.rules`

1. Oversized ICMP Echo Request (>128 Bytes)

Objective

Detect ICMP Echo Requests larger than 128 bytes.

IDS Rule

```
alert icmp any any -> $HOME_NET any (
msg:"LAB-01 Oversized ICMP Echo Request (>128 Bytes)";
itype:8;
dsize:>128;
sid:1000001;
rev:1;
)
```

IPS Rule

```
drop icmp any any -> $HOME_NET any (
msg:"IPS BLOCK: Oversized ICMP Packet";
itype:8;
dsize:>128;
sid:2000001;
rev:1;
)
```

Kali Test

Linux ping: $ping -s 200 192.168.1.10

Using Scapy: $ send(IP(dst="192.168.1.10")/ICMP()/("A"*200))

```
bat /etc/snort/rules/local.rules
```

File: **/etc/snort/rules/local.rules**

```
1   #Detect ICMP Echo Requests larger than 128 bytes
2   #IDS rule
3   alert icmp any any -> $HOME_NET any (msg:"LAB-01 Oversized ICMP Echo Reques
    t (>128 Bytes)"; itype:8; dsize:>128; sid:1000001; rev:1;)
4
```

```
root ~
snort -q -A console -i ens33 -c /etc/snort/snort.conf
07/04-19:15:40.863973  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:15:41.866966  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:15:42.868699  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
```

```
┌──(root㉿kali)-[~]
└─# ping -s 200 192.168.74.135
PING 192.168.74.135 (192.168.74.135) 200(228)
 bytes of data.
208 bytes from 192.168.74.135: icmp_seq=1 ttl
=64 time=1.58 ms
208 bytes from 192.168.74.135: icmp_seq=2 ttl
=64 time=1.13 ms
208 bytes from 192.168.74.135: icmp_seq=3 ttl
=64 time=1.03 ms
208 bytes from 192.168.74.135: icmp_seq=4 ttl
=64 time=0.946 ms
208 bytes from 192.168.74.135: icmp_seq=5 ttl
=64 time=0.946 ms
```

```
root@ubuntu-VMware-Virtual-Platfor...  ubuntu@ubuntu-VMware-Virtual-Plat...  root@ubuntu-VMware-Virtual-Platfor...
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:15:58.969411  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:15:59.971354  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:16:00.972985  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:16:01.974310  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:16:02.990109  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:16:04.014735  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:16:05.016403  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:17:50.045057  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:17:51.047398  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:17:52.075350  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
07/04-19:17:53.076986  [**] [1:1000001:1] LAB-01 Oversized ICMP Echo Request (>128 B
ytes) [**] [Priority: 0] {ICMP} 192.168.74.129 -> 192.168.74.135
```

```
┌──(root㉿kali)-[~]
└─# ping -s 100 192.168.74.135
PING 192.168.74.135 (192.168.74.135) 100(128)
 bytes of data.
108 bytes from 192.168.74.135: icmp_seq=1 ttl
=64 time=1.61 ms
108 bytes from 192.168.74.135: icmp_seq=2 ttl
=64 time=0.639 ms
^C
── 192.168.74.135 ping statistics ──
2 packets transmitted, 2 received, 0% packet
loss, time 1002ms
rtt min/avg/max/mdev = 0.639/1.123/1.607/0.48
4 ms

┌──(root㉿kali)-[~]
└─# ping -s 550 192.168.74.135
PING 192.168.74.135 (192.168.74.135) 550(578)
 bytes of data.
558 bytes from 192.168.74.135: icmp_seq=1 ttl
=64 time=1.61 ms
558 bytes from 192.168.74.135: icmp_seq=2 ttl
=64 time=0.773 ms
558 bytes from 192.168.74.135: icmp_seq=3 ttl
```

2. ICMP Flood (30 packets in 1 minute)

IDS Rule

```
alert icmp any any -> $HOME_NET any ( msg:"LAB-02 ICMP Flood Detected"; itype:8; detection_filter:track by_src,count 30,seconds 60; sid:1000002; rev:1; )
```

IPS Rule

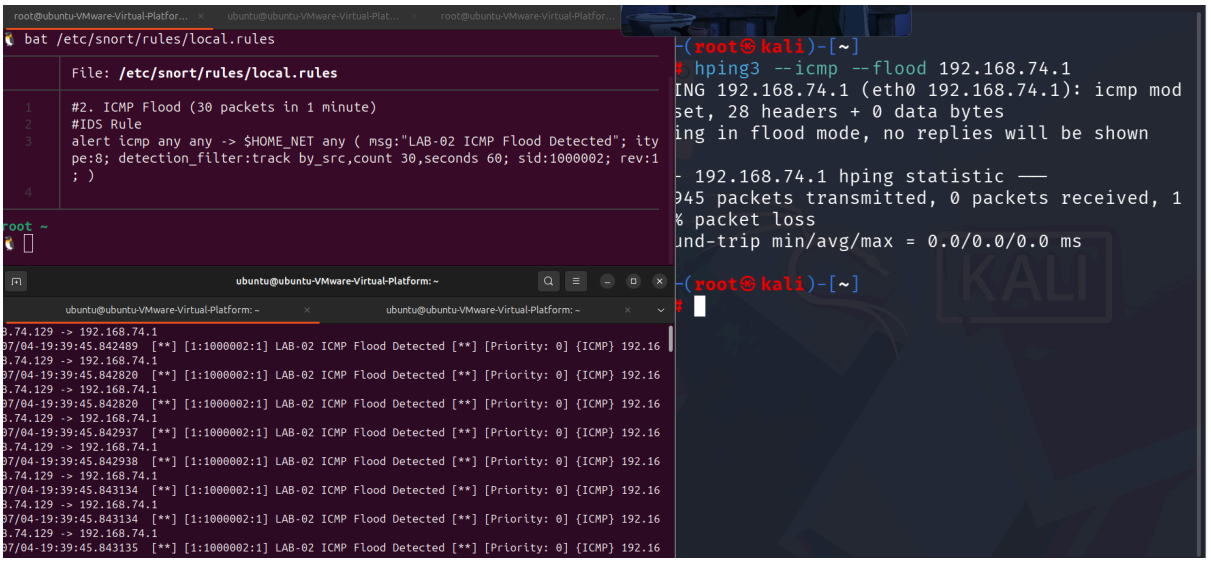
```
drop icmp any any -> $HOME_NET any (
msg:"IPS BLOCK: ICMP Flood";
itype:8;
detection_filter:track by_src,count 30,seconds 60;
sid:2000002;
rev:1;
)
```

Kali Test

ping -f 192.168.1.10
or
hping3 --icmp --flood 192.168.1.10

Scapy

send(IP(dst="192.168.1.10")/ICMP(),count=40)



3. TCP SYN Flood

IDS Rule

```
alert tcp any any -> $HOME_NET 80 ( msg:"LAB-03 TCP SYN Flood";
flags:S; detection_filter:track by_src,count 20,seconds 60; sid:1000003;
rev:1; )
```

IPS Rule

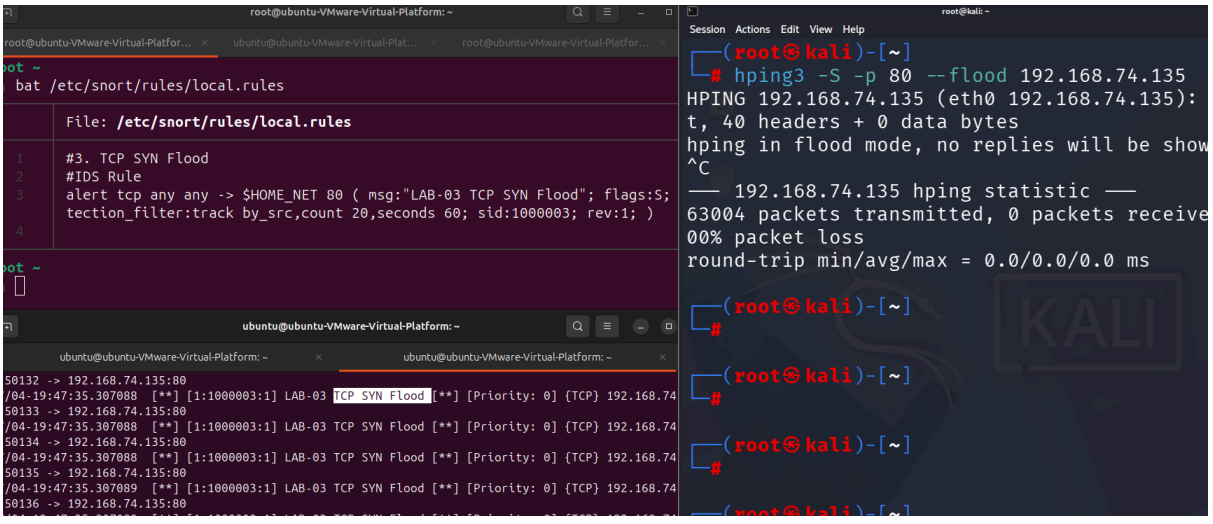
```
drop tcp any any -> $HOME_NET 80 (
msg:"IPS BLOCK: TCP SYN Flood";
flags:S;
detection_filter:track by_src,count 20,seconds 60;
sid:2000003;
rev:1;
)
```

Kali Test

hping3 -S -p 80 --flood 192.168.1.10

Scapy

send(IP(dst="192.168.1.10")/TCP(dport=80,flags="S"),count=30)



4. SSH Brute Force

IDS Rule

```
alert tcp any any -> $HOME_NET 22 ( \
    msg:"LAB-04 SSH Brute Force Attempt"; \
    detection_filter:track by_src, count 10, seconds 60; \
    sid:1000004; \
    rev:1;\
)
```

IPS Rule

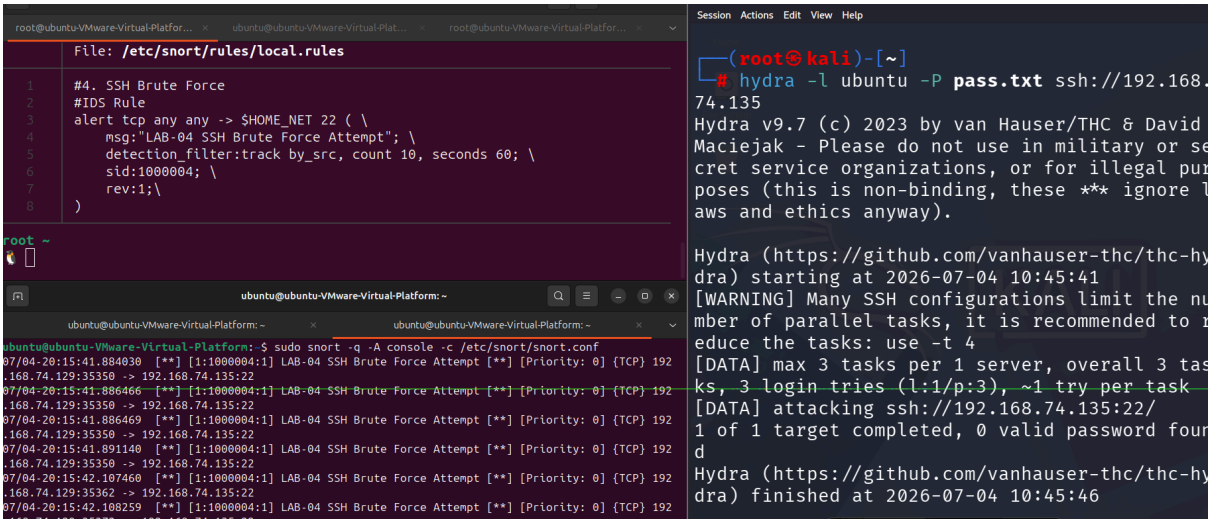
```
drop tcp any any -> $HOME_NET 22 (
msg:"IPS BLOCK: SSH Brute Force";
flags:S;
detection_filter:track by_src,count 10,seconds 60;
sid:2000004;
rev:1;
)
```

Kali Test

hydra -l root -P rockyou.txt ssh://192.168.1.10

or

for i in {1..20}
do
ssh fake@192.168.1.10
done



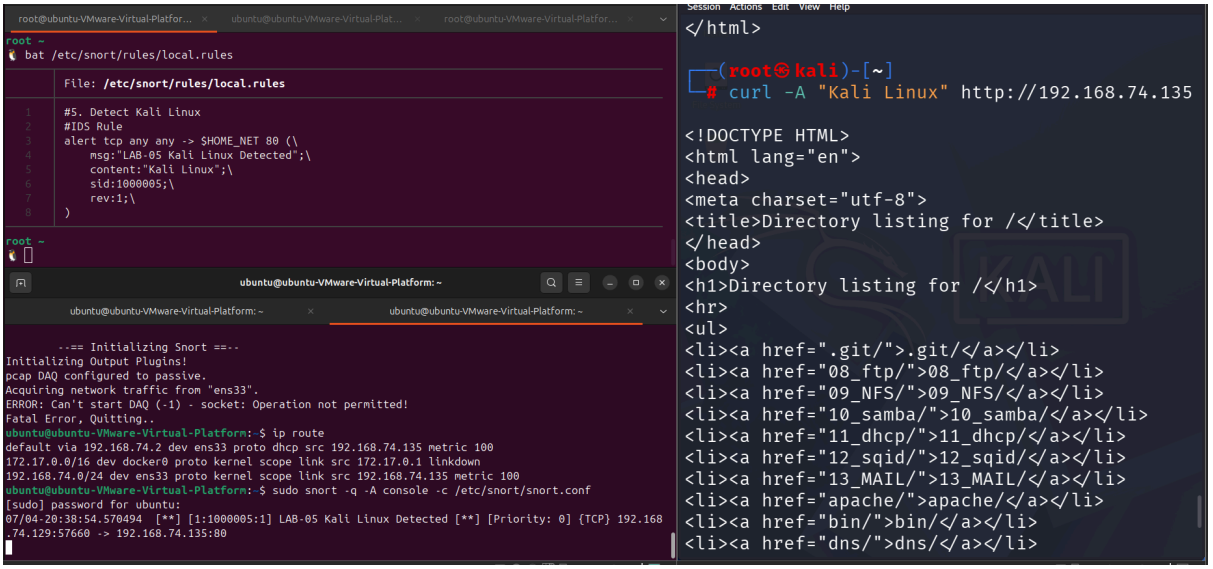
5. Detect Kali Linux

IDS Rule

```
alert http any any -> any any (  \
msg:"LAB-05 Possible Kali Linux User-Agent";  \
content:"Kali Linux";  \
sid:1000005;  \
rev:1; \
)
```

Test

curl -A "Kali Linux" http://192.168.1.10



6. Gmail Access

IDS Rule

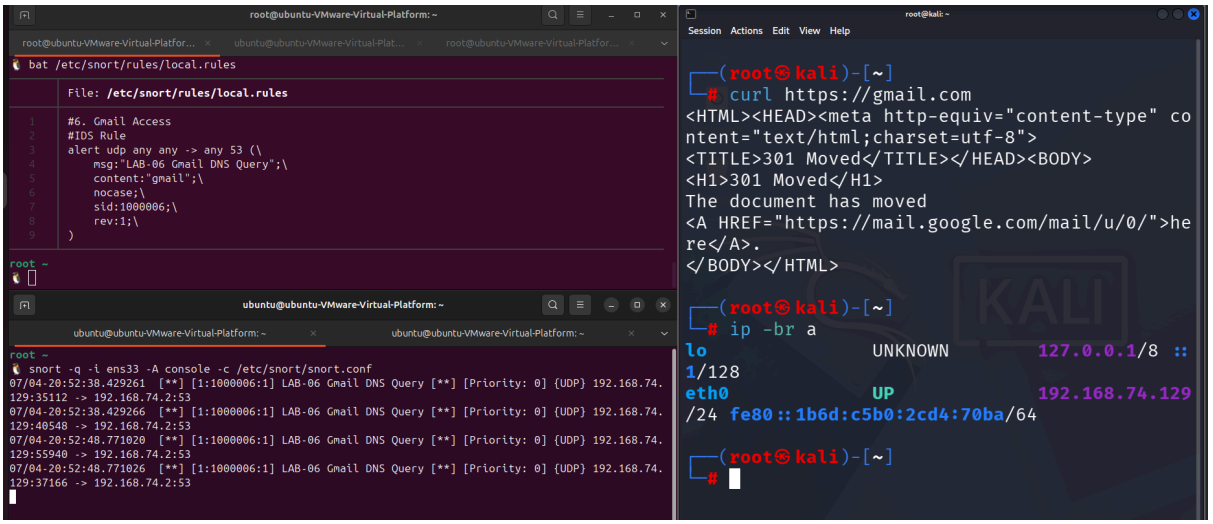
```
alert udp any any -> any 53 (\
   msg:"LAB-06 Gmail DNS Query";\
   content:"gmail";\
   nocase;\
   sid:1000006;\
   rev:1;\
)
```

Test

firefox https://gmail.com

or

curl https://gmail.com



7. Facebook Access

```
alert udp any any -> any 53 (\
   msg:"LAB-06 Gmail DNS Query";\
   content:"Facebook";\
   nocase;\
   sid:1000006;\
   rev:1;\
)
```

Test

curl https://facebook.com

8. Instagram Access

```
alert udp any any -> any 53 (\
   msg:"LAB-06 Gmail DNS Query";\
   content:"instagram";\
   nocase;\
   sid:1000006;\
   rev:1;\
)
```

Test

curl https://instagram.com

9. Port Scan

IDS Rule

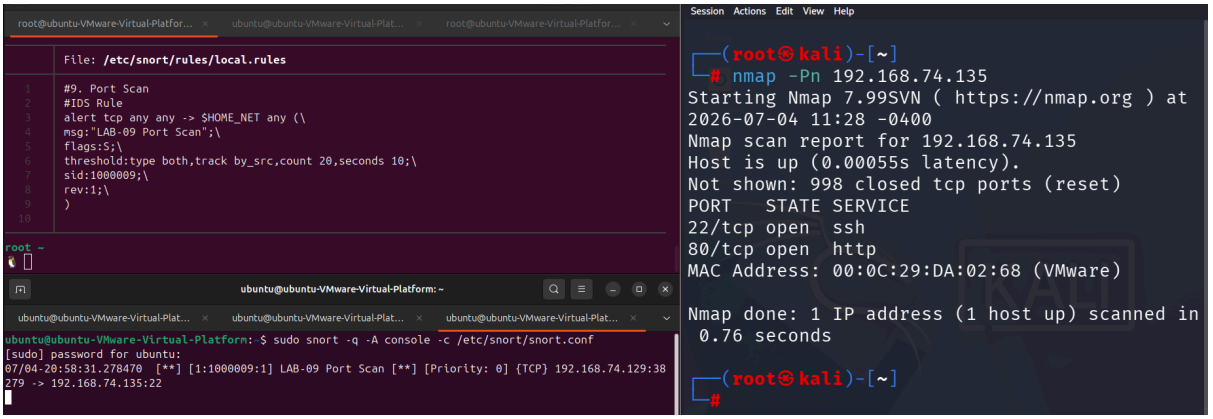
```
alert tcp any any -> $HOME_NET any (\
msg:"LAB-09 Port Scan";\
flags:S;\
threshold:type both,track by_src,count 20,seconds 10;\
sid:1000009;\
rev:1;\
)
```

Kali Test

nmap -Pn 192.168.1.10

or

nmap -p- 192.168.1.10



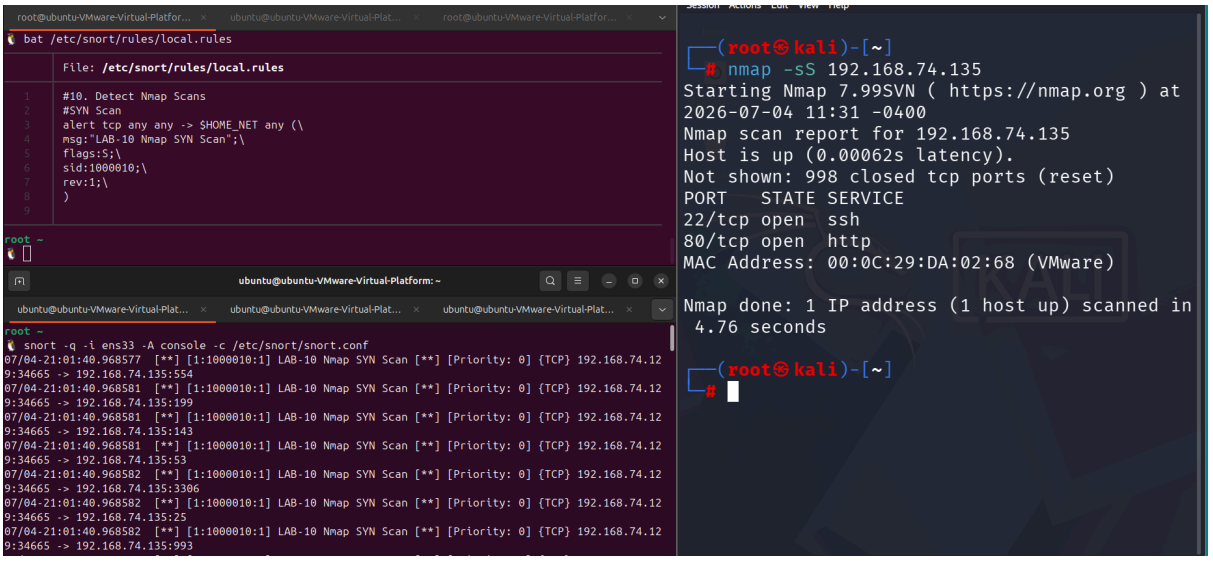
10. Detect Nmap Scans

SYN Scan

nmap -sS 192.168.1.10

Rule

```
alert tcp any any -> $HOME_NET any (\
msg:"LAB-10 Nmap SYN Scan";\
flags:S;\
sid:1000010;\
rev:1;\
)
```



FIN Scan

nmap -sF 192.168.1.10

Rule

```
alert tcp any any -> $HOME_NET any (
msg:"LAB-10 Nmap FIN Scan";
flags:F;
sid:1000011;
rev:1;
)
```

NULL Scan

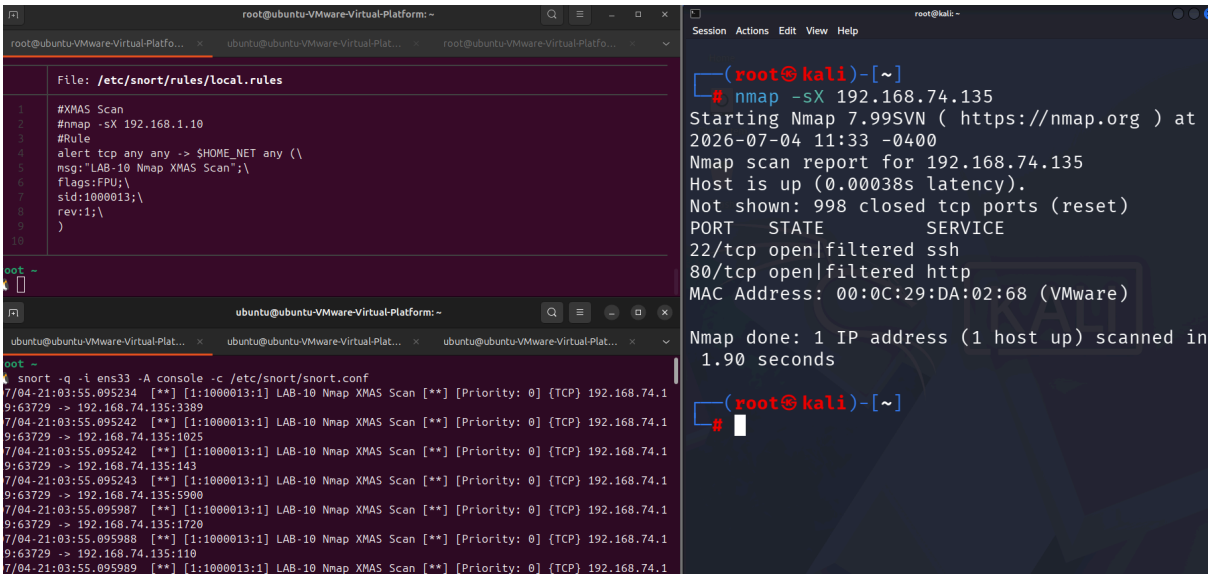
nmap -sN 192.168.1.10

```
alert tcp any any -> $HOME_NET any (
msg:"LAB-10 Nmap NULL Scan";
flags:0;
sid:1000012;
rev:1;
)
```

XMAS Scan

nmap -sX 192.168.1.10

```
alert tcp any any -> $HOME_NET any (\
msg:"LAB-10 Nmap XMAS Scan";\
flags:FPU;\
sid:1000013;\
rev:1;\
)
```



Version Detection

nmap -sV 192.168.1.10

OS Detection

nmap -O 192.168.1.10

11. SQL Injection

IDS Rule

```
alert tcp any any -> $HOME_NET 80 (\
msg:"LAB-11 SQL Injection";\
content:"' OR 1=1";\
nocase;\
http_uri;\
sid:1000014;\
rev:1;\
)
```

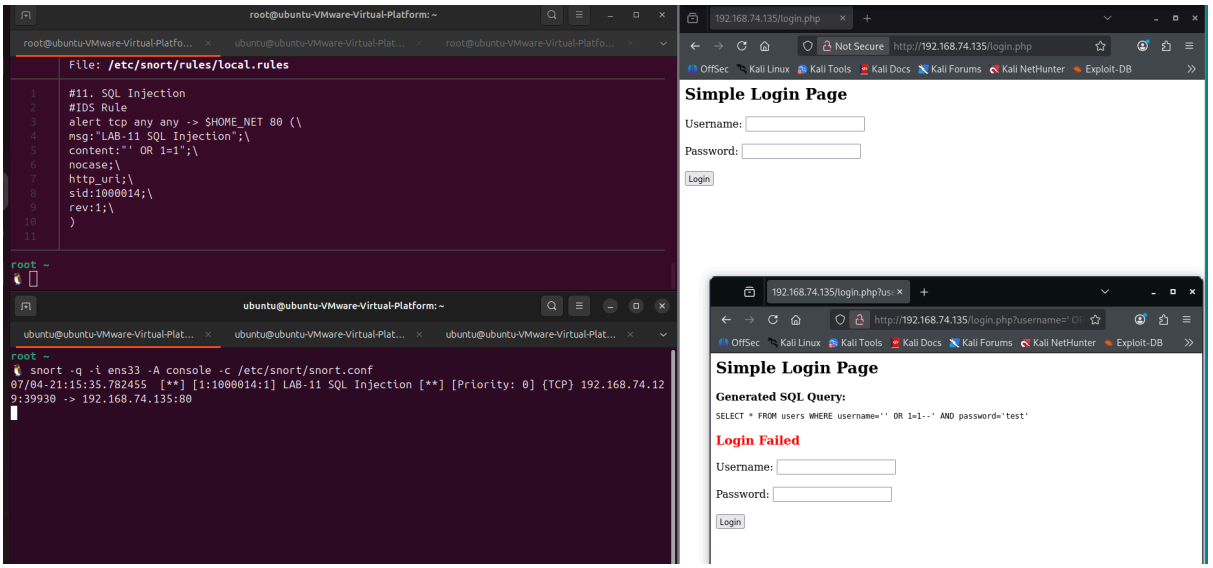
Kali Test

Browser

http://192.168.1.10/login.php?user=admin' OR 1=1--

or

sqlmap -u "http://192.168.1.10/login.php?id=1"



1. Install Apache + PHP

```
sudo apt update
sudo apt install apache2 php -y
```

---

2. Create the login page

```
sudo nano /var/www/html/login.php
```

Paste:

```
<?php
$username = $_GET['username'] ?? '';
$password = $_GET['password'] ?? '';

echo "<h2>Simple Login Page</h2>";

if ($username != "" || $password != "") {

  $query = "SELECT * FROM users WHERE username='$username' AND password='$password'";

  echo "<b>Generated SQL Query:</b><br>";
  echo "<pre>$query</pre>";

  if ($username == "admin" && $password == "admin123") {
     echo "<h3 style='color:green;'>Login Successful</h3>";
  } else {
     echo "<h3 style='color:red;'>Login Failed</h3>";
  }
}
?>

<form method="GET">
Username:
<input type="text" name="username"><br><br>

Password:
<input type="password" name="password"><br><br>

<input type="submit" value="Login">
</form>
```

---

3. Restart Apache

```
sudo systemctl restart apache2
```

---

4. Open from Kali

http://192.168.74.135/login.php

---

5. Test SQL Injection

Normal request

http://192.168.74.135/login.php?username=admin&password=admin123

SQL Injection

http://192.168.74.135/login.php?username=' OR 1=1--&password=test

or

curl "http://192.168.74.135/login.php?username=' OR 1=1--&password=test"

12. Cross Site Scripting

IDS Rule

```
alert http any any -> $HOME_NET 80 (
msg:"LAB-12 XSS Attack";
content:"<script>";
nocase;
http_uri;
sid:1000015;
rev:1;
)
```

Kali Test

http://192.168.1.10/search.php?q=<script>alert(1)</script>

or

curl "http://192.168.1.10/?q=<script>alert(1)</script>"

13. Directory Traversal

IDS Rule

```
alert http any any -> $HOME_NET 80 (
msg:"LAB-13 Directory Traversal";
content:"../";
http_uri;
sid:1000016;
rev:1;
)
```

Kali Test

http://192.168.1.10/../../etc/passwd

or

curl "http://192.168.1.10/../../etc/passwd"

Encoded test

curl "http://192.168.1.10/%2e%2e/%2e%2e/etc/passwd"

Recommended Lab Flow

For each exercise, document:

1. **Objective**
2. **Attack Description**
3. **Snort Rule**
4. **Rule Explanation**

Restart Snort
sudo systemctl restart snort

5.
6. **Run the Kali attack**

Capture the alert
sudo tail -f /var/log/snort/alert

7.
8. **Take screenshots** of:
    - Attack command on Kali
    - Snort alert
    - Wireshark packet (optional)
9. **Result**
10. **Conclusion**

This structure is suitable for a practical Snort IDS/IPS lab report and covers both the detection (IDS) and prevention (IPS) objectives in your assignment.

Here's what each of those Snort rule options means, with simple examples.

Attribute	Purpose	Example
msg	Custom alert message displayed when the rule matches.	`msg:"ICMP Flood Detected";`
sid	**Signature ID**. A unique identifier for the rule. Custom rules usually use IDs above `1000000`.	`sid:1000001;`
rev	Rule revision number. Increment it whenever you modify the rule.	`rev:2;`
itype	Matches a specific **ICMP Type**.	`itype:8;` → ICMP Echo Request (Ping)
dsize	Matches the packet payload size.	`dsize:>128;` → Payload larger than 128 bytes
detection_filter:track by_src,count 30,seconds 60;	Detects traffic rate. Generates an alert when **one source IP** sends **30 or more packets within 60 seconds**.	Used for ICMP floods, brute-force, SYN floods
flags:F	Match TCP packets with only the **FIN** flag set.	`flags:F;`

	Often indicates a FIN scan.	
flags:P	Match packets with the **PSH (Push)** flag.	`flags:P;`
flags:U	Match packets with the **URG (Urgent)** flag.	`flags:U;`
flags:FPU	Match packets with **FIN + PSH + URG** flags (XMAS scan).	`flags:FPU;`
http_header	Restricts the search to the **HTTP headers** only.	`content:"Kali"; http_header;`
tls.sni	Searches the **Server Name Indication (SNI)** in a TLS (HTTPS) ClientHello before encryption.	`content:"gmail.com"; tls.sni;`
content	Searches for a specific text or byte sequence in the packet.	`content:"facebook.com";`
threshold:type both,track by_src,count 20,seconds 10;	Limits alerts. If a source sends **20 matching packets in 10 seconds**, an alert is generated while suppressing	Port scan detection

	excessive duplicates. Useful for scans.	
content:"' OR 1=1";	Looks for a common **SQL Injection** payload in HTTP traffic.	Detects SQLi attempts
nocase	Makes the preceding `content` match **case-insensitively**.	Matches `ADMIN`, `Admin`, and `admin`
content:"";	Detects a common **Cross-Site Scripting (XSS)** payload.	`<script>alert(1)</script>`
content:"../";	Detects a **Directory Traversal** attempt.	`../../etc/passwd`
http_uri	Restricts the `content` search to the **HTTP request URI (URL path/query)**.	Detects `../` only in the requested URL

Difference: `detection_filter` vs `threshold`

- `detection_filter`: Detects **high-rate attacks** (e.g., flood, brute force). It alerts **after** the specified number of packets within the time window.
- `threshold`: Controls **how often alerts are generated** to avoid log flooding when many matching packets occur.

Common TCP Flags

Flag	Meaning	Used By

`S`	SYN	SYN Scan, connection initiation
`A`	ACK	Established connections
`F`	FIN	FIN Scan
`R`	RST	Connection reset
`P`	PSH	Push data immediately
`U`	URG	Urgent data
`FPU`	FIN + PSH + URG	XMAS Scan
`0`	No flags	NULL Scan

These are the attributes you'll use most often for your Snort IDS/IPS lab.